complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + findfirst() conditional check before first match

filter() null checks + findfirst() conditional check before first match

filter() comparisons + findfirst() conditional check before first match

filter() method calls returning boolean + findfirst() conditional check before first match

distinct() rely on equals() / hashcode() + findfirst() conditional check before first match

distinct() immutable objects + findfirst() conditional check before first match

map() transform objects + findfirst() conditional check before first match

map() create new objects + findfirst() conditional check before first match

map() call pure functions + findfirst() conditional check before first match

flatmap() return stream + findfirst() conditional check before first match

flatmap() split data + findfirst() conditional check before first match

flatmap() expand collections + findfirst() conditional check before first match

sorted() natural ordering + findfirst() conditional check before first match

sorted() custom comparator + findfirst() conditional check before first match

sorted() multi-field compare + findfirst() conditional check before first match

peek() logging + findfirst() conditional check before first match

peek() debugging + findfirst() conditional check before first match

peek() metrics + findfirst() conditional check before first match

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = findFirst()**
- Purpose = **conditional check before the first matching element**
- Demonstrates **short-circuiting** (stream stops after first match)
- Java **8+ compatible**
- No collectors, no reduce, no extra logic

 findFirst() returns Optional<T> and respects **encounter order**.

---

1 filter() boolean checks + findFirst()

```
import java.util.*;

public class FilterBooleanFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstEven =
            List.of(11, 14, 18).stream()
                  .filter(n -> n % 2 == 0)
                  .findFirst();

      System.out.println(firstEven.orElse(-1));
   }
}
```

---

2 filter() null checks + findFirst()

```
import java.util.*;

public class FilterNullFindFirst {
   public static void main(String[] args) {
      Optional<String> firstNonNull =
            Arrays.asList(null, "Anurag", "Amit").stream()
                  .filter(Objects::nonNull)
                  .findFirst();

      System.out.println(firstNonNull.orElse("NONE"));
   }
}
```

---

3 filter() comparisons + findFirst()

```
import java.util.*;

public class FilterComparisonFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstHighSalary =
            List.of(20000, 45000, 60000).stream()
                  .filter(s -> s >= 40000)
                  .findFirst();

      System.out.println(firstHighSalary.orElse(0));
   }
}
```

---

4 filter() method call returning boolean + findFirst()

```
import java.util.*;

public class FilterMethodFindFirst {

   static class User {
      String name;
      boolean active;
      User(String name, boolean active) {
         this.name = name; this.active = active;
      }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      Optional<User> firstActive =
            List.of(
                  new User("A", false),
                  new User("B", true),
                  new User("C", true)
            ).stream()
             .filter(User::isActive)
             .findFirst();

      System.out.println(firstActive.map(u -> u.name).orElse("NONE"));
   }
}
```

---

5 distinct() rely on equals() / hashCode() + findFirst()

```
import java.util.*;

public class DistinctEqualsFindFirst {

   static class Person {
      String name;
      Person(String name) { this.name = name; }
      @Override public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }
      @Override public int hashCode() { return Objects.hash(name); }
   }

   public static void main(String[] args) {
      Optional<Person> firstUnique =
            List.of(
                  new Person("Anurag"),
                  new Person("Anurag"),
                  new Person("Ravi")
            ).stream()
             .distinct()
             .findFirst();

      System.out.println(firstUnique.map(p -> p.name).orElse("NONE"));
   }
}
```

---

6 distinct() immutable objects + findFirst()

```
import java.util.*;

public class DistinctImmutableFindFirst {
   public static void main(String[] args) {
      Optional<String> firstUnique =
            List.of("Java", "Java", "Go").stream()
                  .distinct()
                  .findFirst();

      System.out.println(firstUnique.orElse("NONE"));
   }
}
```

---

7 map() transform objects + findFirst()

```
import java.util.*;

public class MapTransformFindFirst {
   public static void main(String[] args) {
      Optional<String> firstUpper =
            List.of("anurag", "amit").stream()
                  .map(String::toUpperCase)
                  .findFirst();

      System.out.println(firstUpper.orElse("NONE"));
   }
}
```

---

8 map() create new objects + findFirst()

```
import java.util.*;

public class MapCreateObjectFindFirst {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      Optional<UserDTO> firstDTO =
            List.of("Anurag", "Amit").stream()
                  .map(UserDTO::new)
                  .findFirst();

      System.out.println(firstDTO.map(d -> d.name).orElse("NONE"));
   }
}
```

---

9 map() call pure functions + findFirst()

```
import java.util.*;

public class MapPureFunctionFindFirst {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      Optional<Integer> firstSquare =
            List.of(2, 3, 4).stream()
                  .map(MapPureFunctionFindFirst::square)
                  .findFirst();

      System.out.println(firstSquare.orElse(0));
   }
}
```

---

10 flatMap() return Stream + findFirst()

```
import java.util.*;

public class FlatMapStreamFindFirst {
   public static void main(String[] args) {
      Optional<Integer> first =
            List.of(
                  List.of(10, 20),
                  List.of(30)
            ).stream()
             .flatMap(List::stream)
             .findFirst();

      System.out.println(first.orElse(0));
   }
}
```

---

11 flatMap() split data + findFirst()

```
import java.util.*;

public class FlatMapSplitFindFirst {
   public static void main(String[] args) {
      Optional<String> firstWord =
            List.of("Java Stream", "Flat Map").stream()
                  .flatMap(s -> Arrays.stream(s.split(" ")))
                  .findFirst();

      System.out.println(firstWord.orElse("NONE"));
   }
}
```

---

12 flatMap() expand collections + findFirst()

```
import java.util.*;

public class FlatMapExpandFindFirst {
   public static void main(String[] args) {
      Optional<String> firstValue =
            Map.of(
                  "A", List.of("1", "2"),
                  "B", List.of("3")
            ).values().stream()
             .flatMap(Collection::stream)
             .findFirst();

      System.out.println(firstValue.orElse("NONE"));
   }
}
```

---

13 sorted() natural ordering + findFirst()

```
import java.util.*;

public class SortedNaturalFindFirst {
   public static void main(String[] args) {
      Optional<Integer> smallest =
            List.of(5, 1, 3).stream()
                  .sorted()
                  .findFirst();

      System.out.println(smallest.orElse(0));
   }
}
```

---

14 sorted() custom comparator + findFirst()

```
import java.util.*;

public class SortedCustomFindFirst {
   public static void main(String[] args) {
      Optional<String> firstDesc =
            List.of("Anurag", "Ravi", "Amit").stream()
                  .sorted(Comparator.reverseOrder())
                  .findFirst();

      System.out.println(firstDesc.orElse("NONE"));
   }
}
```

---

15 sorted() multi-field compare + findFirst()

```
import java.util.*;

public class SortedMultiFieldFindFirst {

   static class Employee {
      String name;
      int age;
      Employee(String name, int age) {
         this.name = name; this.age = age;
      }
   }

   public static void main(String[] args) {
      Optional<Employee> first =
            List.of(
                  new Employee("Anurag", 30),
                  new Employee("Anurag", 25),
                  new Employee("Ravi", 35)
            ).stream()
             .sorted(Comparator
                   .comparing((Employee e) -> e.name)
                   .thenComparingInt(e -> e.age))
             .findFirst();

      System.out.println(first.map(e -> e.name + " " + e.age).orElse("NONE"));
   }
}
```

---

16 peek() logging + findFirst()

```
import java.util.*;

public class PeekLoggingFindFirst {
   public static void main(String[] args) {
      Optional<Integer> first =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Peek: " + n))
                  .findFirst();

      System.out.println(first.orElse(0));
   }
}
```

---

17 peek() debugging + findFirst()

```
import java.util.*;

public class PeekDebugFindFirst {
   public static void main(String[] args) {
      Optional<Integer> first =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit");
                  })
                  .findFirst();

      System.out.println(first.orElse(0));
   }
}
```

---

18 peek() metrics + findFirst()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsFindFirst {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      Optional<Integer> first =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .findFirst();

      System.out.println("First: " + first.orElse(0));
      System.out.println("Processed elements: " + counter.get());
   }
}
```